

Laboratorio #1 — Reporte Individual

Introducción a Wireshark

1. Datos

- Javier López — [Carnet]

2. Introducción

Esta segunda parte del laboratorio consistió en instalar Wireshark, personalizar el entorno de trabajo, configurar una captura con ring buffer y analizar tráfico HTTP real. El objetivo era familiarizarse con la herramienta y, sobre todo, ver de primera mano cómo viaja la información: qué protocolos aparecen, qué información lleva cada paquete y qué se puede deducir de una conversación cliente-servidor solo mirando los bytes que pasan por la interfaz.

3. Personalización del entorno

3.1 Perfil de configuración

Se creó un perfil personal con nombre y apellido. Los perfiles en Wireshark permiten guardar de forma independiente las columnas, reglas de color, filtros y layout, de manera que uno puede tener configuraciones distintas para distintos tipos de análisis sin sobrescribir la configuración por defecto.

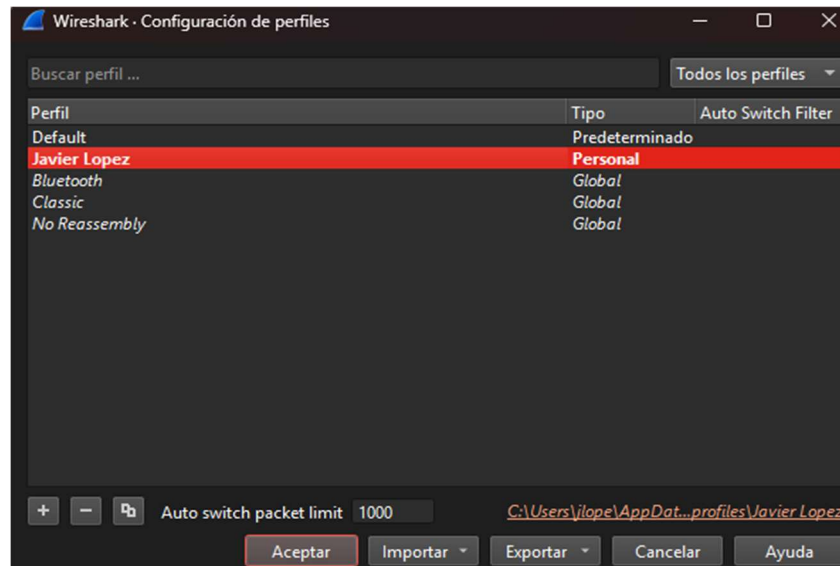


Figura 1: Perfil personal "Javier Lopez" creado en Edit → Configuration Profiles.

3.2 Columnas y formato de tiempo

Se aplicó el formato de tiempo Time of Day (View → Time Display Format), se agregó una columna personalizada con la longitud del protocolo y se ocultó la columna Length que viene por defecto.

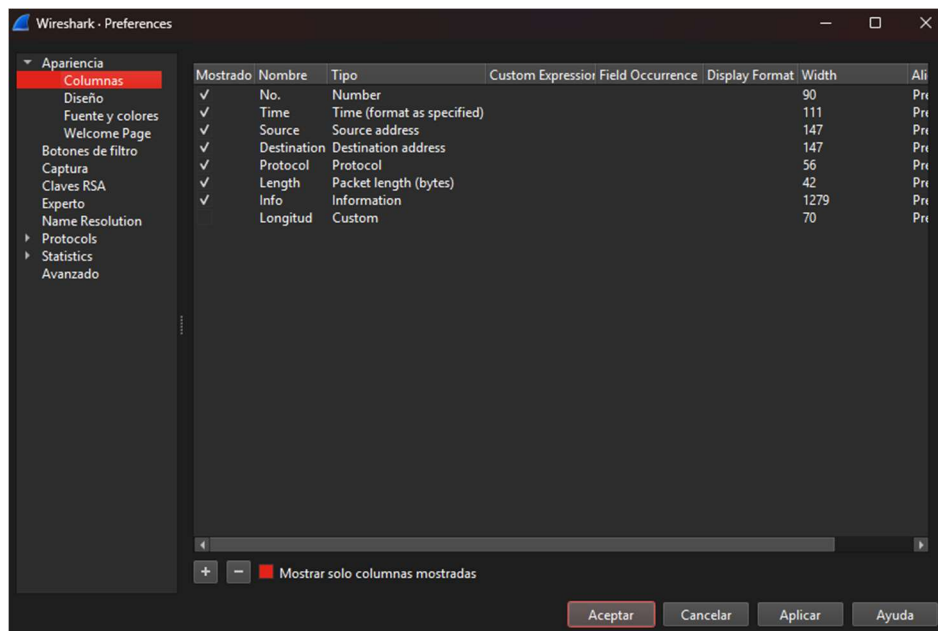


Figura 2: Configuración de columnas. Se observa la columna personalizada agregada y la columna Length desmarcada.

3.3 Layout de paneles

Se cambió el esquema de paneles por defecto por una distribución distinta, en la que el listado de paquetes ocupa la parte superior y los detalles y bytes se muestran debajo. La ventaja de este layout es que da más espacio horizontal a la columna Info, que suele ser la que uno más lee al hacer un barrido rápido de la captura.

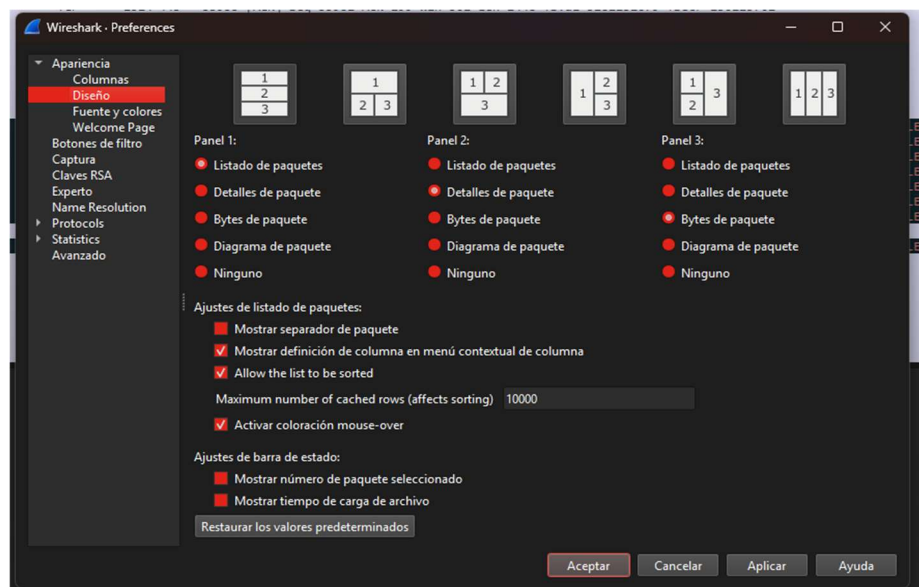


Figura 3: Esquema de paneles personalizado (Preferences → Apariencia → Diseño).

3.4 Regla de color y botón de filtro

Se creó una regla de color para resaltar los paquetes TCP cuya bandera SYN esté activa, es decir, los intentos de establecimiento de conexión. El filtro utilizado fue:

```
tcp.flags.syn == 1
```

La regla se colocó en la primera posición de la lista, ya que Wireshark aplica la primera regla que coincide con el paquete; si se dejara debajo de las reglas por defecto de TCP, estas la sobrescribirían y el color nunca se vería.

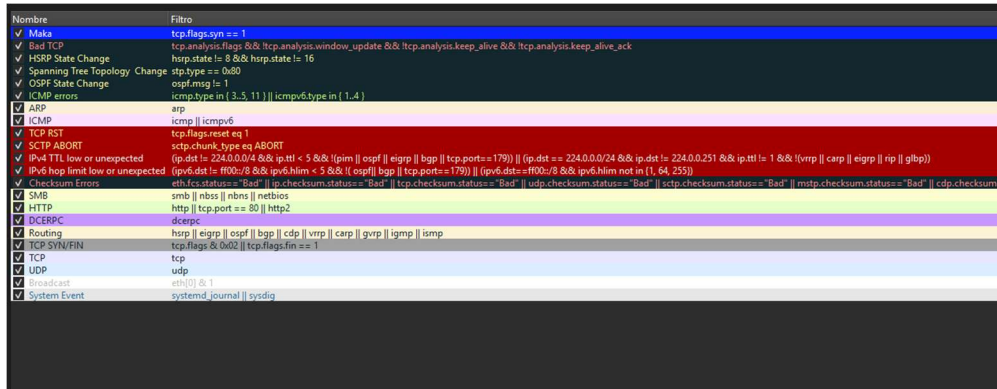


Figura 4: Regla de color creada en View → Coloring Rules, colocada al inicio de la lista para que tenga prioridad.

Adicionalmente se creó un botón de filtro en la esquina superior derecha con ese mismo filtro, lo que permite aislar los paquetes SYN con un solo clic en lugar de escribir el filtro cada vez.

3.5 Ocultamiento de interfaces virtuales

Se desactivaron las interfaces virtuales desde Capture → Options, dejando visibles únicamente las interfaces físicas (Ethernet y WiFi). Esto simplifica la pantalla de inicio y evita capturar tráfico de adaptadores virtuales que no aportan al análisis.

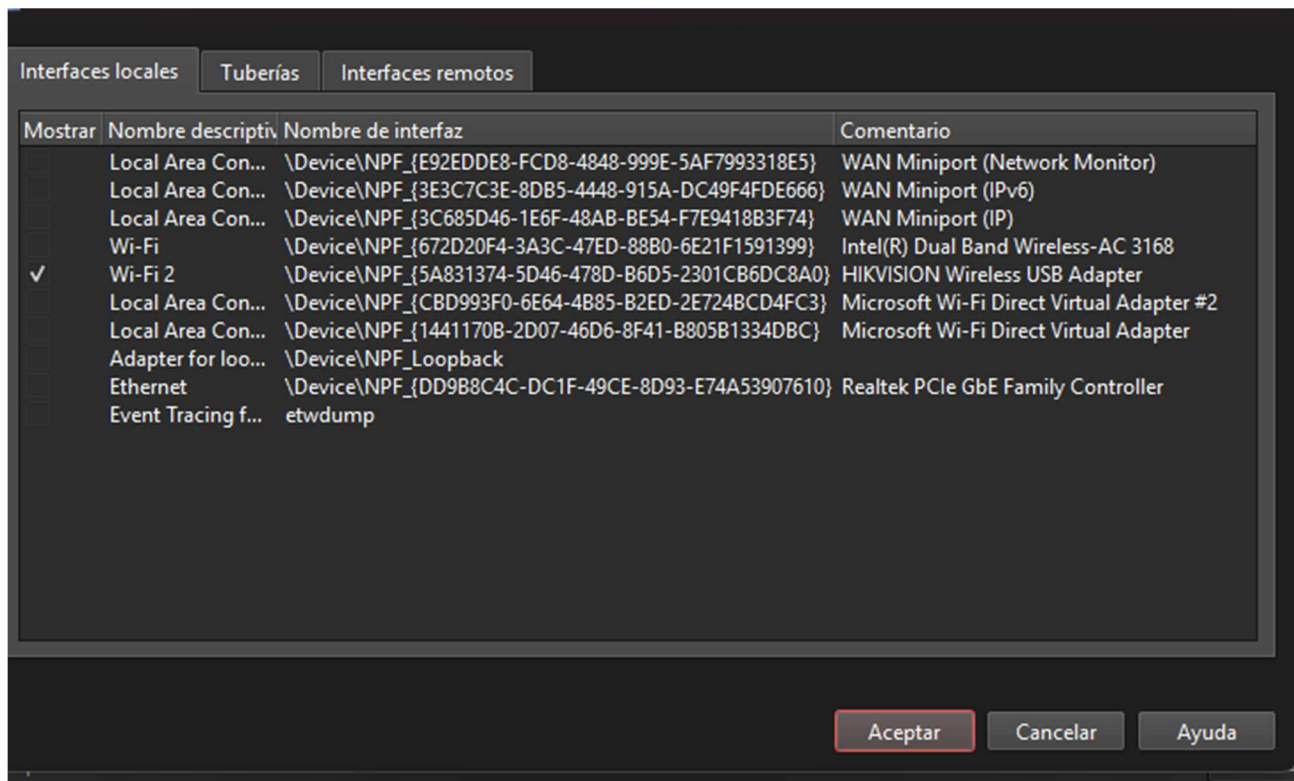


Figura 5: Lista de interfaces de captura tras ocultar las virtuales.

A continuación se muestra el entorno final ya personalizado, con el archivo intro-wireshark-trace1.pcap abierto:

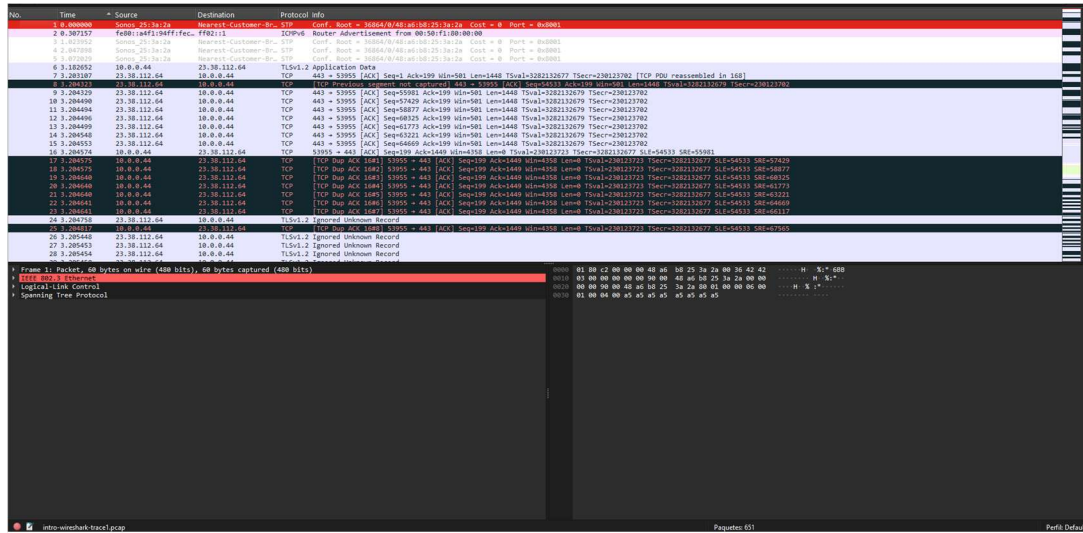


Figura 6: Entorno final con el archivo intro-wireshark-trace1.pcap cargado (651 paquetes).

4. Configuración de la captura de paquetes

4.1 ifconfig / ipconfig

Se ejecutó el comando correspondiente en la terminal para inspeccionar las interfaces de red del equipo. El comando lista cada adaptador disponible junto con su información de configuración:

- Dirección IPv4 y máscara de subred: identifican al host dentro de la red local y determinan qué parte de la dirección corresponde a la red y cuál al host.
- Dirección IPv6: direccionamiento de 128 bits; suele aparecer una dirección link-local (fe80::) que solo tiene validez dentro del segmento local.
- Dirección MAC (physical address): identificador de capa 2, único por adaptador, usado para la entrega dentro de la red local.
- Puerta de enlace predeterminada: el router al que se envía todo el tráfico cuyo destino está fuera de la red local.
- Estado de la interfaz: si está activa (UP) o inactiva, y estadísticas de paquetes transmitidos y recibidos.

Además de las interfaces físicas, aparecen interfaces virtuales como loopback (127.0.0.1), adaptadores de máquinas virtuales o de VPN. Estas son las que conviene desactivar en Wireshark, ya que no transportan el tráfico real que se quiere analizar.

```
Adaptador de LAN inalámbrica Wi-Fi 2:

Sufijo DNS específico para la conexión. . . :
Dirección IPv6 . . . . . : 2800:98:1614:b975:653c:cf40:19f:2012
Dirección IPv6 temporal. . . . . : 2800:98:1614:b975:b949:3c52:4746:7be5
Vínculo: dirección IPv6 local. . . : fe80::2cd7:76c2:d578:c086%10
Dirección IPv4. . . . . : 192.168.1.25
Máscara de subred . . . . . : 255.255.255.0
Puerta de enlace predeterminada . . . . . : fe80::1%10
                                           192.168.1.1

Adaptador de LAN inalámbrica Wi-Fi:
```

Figura 7: Salida del comando de configuración de red.

4.2 Ring buffer

Se configuró una captura con ring buffer sobre la interfaz física, con un tamaño de 5 MB por archivo y un máximo de 10 archivos, nombrados con el formato lab1_carnet.pcap. El ring buffer funciona como una cola circular: cuando el archivo actual alcanza los 5 MB, Wireshark abre uno nuevo, y al llegar al décimo archivo comienza a sobrescribir el más antiguo.

La utilidad de esto es poder dejar una captura corriendo durante horas sin llenar el disco: se conserva siempre una ventana móvil del tráfico más reciente, que es justamente lo que interesa cuando se está esperando a que se reproduzca un problema intermitente.

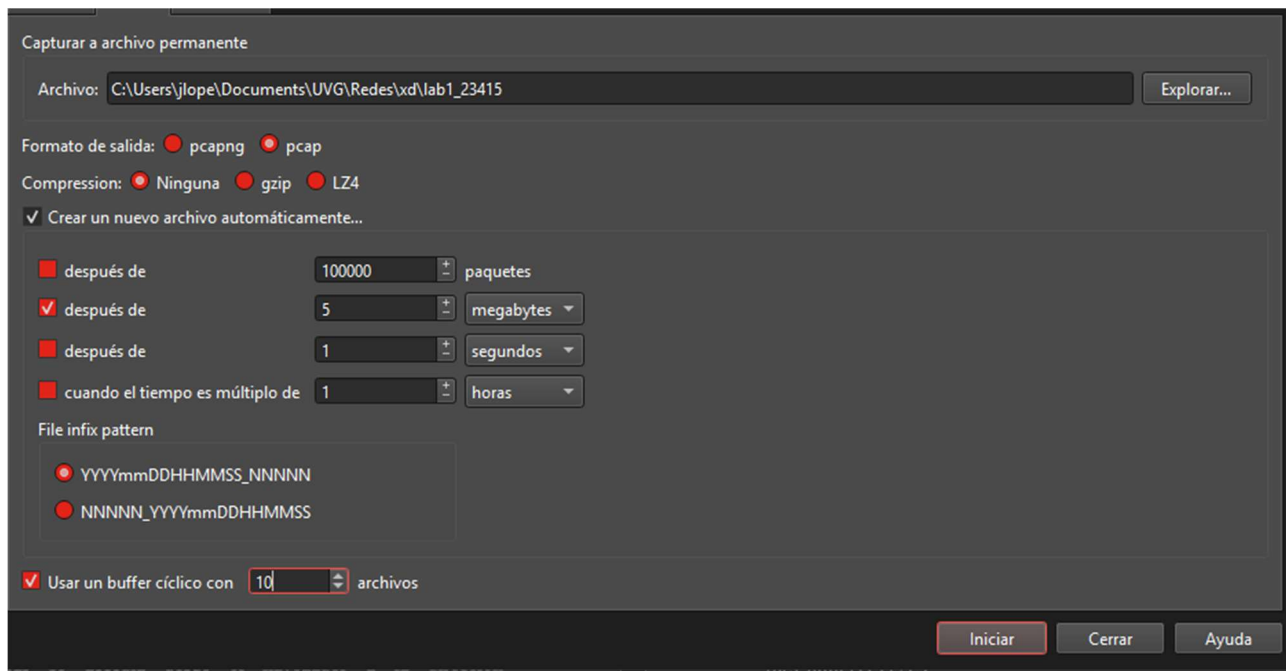


Figura 8: Configuración del ring buffer (5 MB por archivo, máximo 10 archivos).

5. Análisis de paquetes HTTP

Se realizó una captura sin filtro mientras se accedía desde el navegador a la dirección <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>. Posteriormente se aplicó el filtro `http` para aislar la conversación relevante. En la captura se identifican el GET del cliente (192.168.1.25) hacia el servidor (128.119.245.12) y la respuesta 200 OK correspondiente.

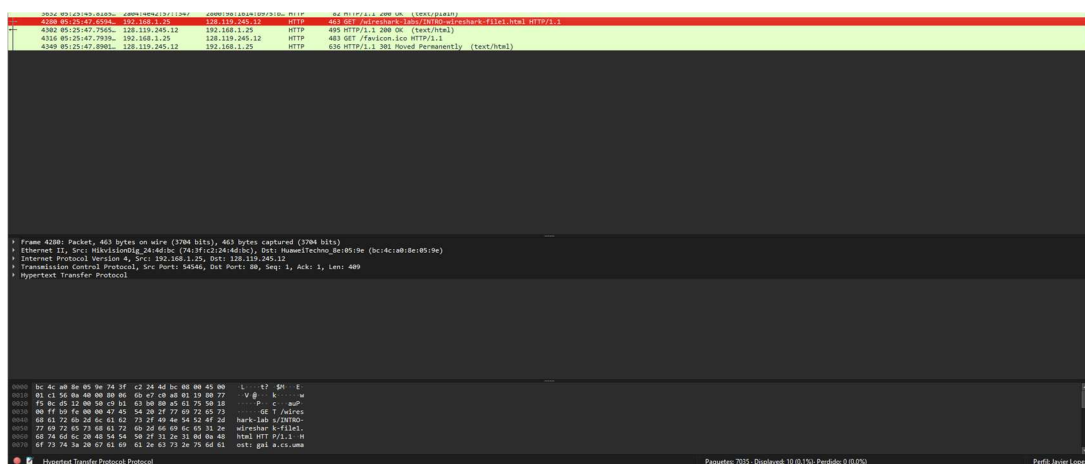


Figura 10: Paquete 4280 — GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1 enviado por el navegador al puerto 80 del servidor.

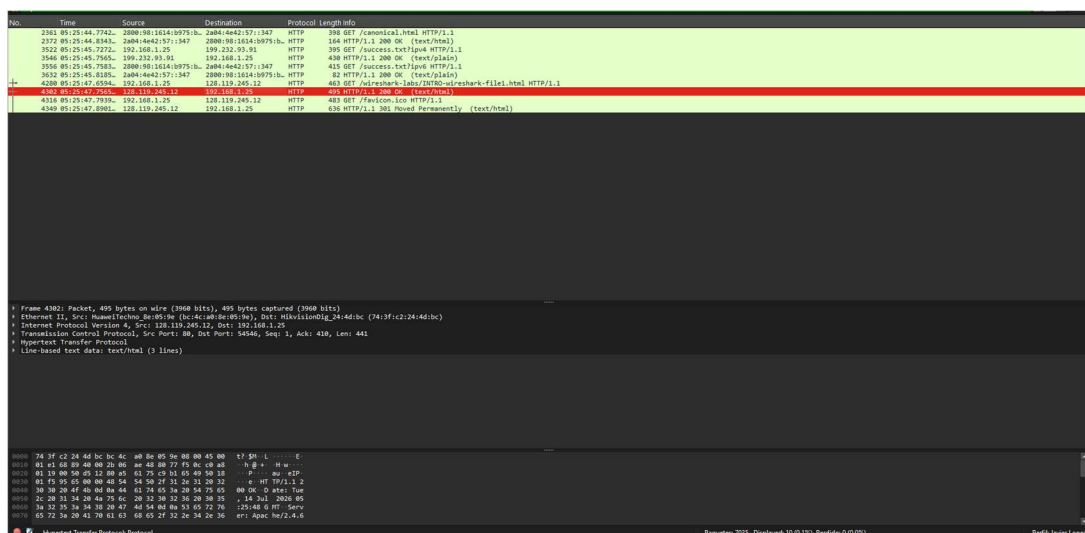


Figura 11: Paquete 4302 — respuesta HTTP/1.1 200 OK del servidor (Apache/2.4.6), con contenido text/html.

a. ¿Qué versión de HTTP está ejecutando el navegador?

HTTP/1.1. Se observa en la línea de solicitud del paquete 4280: GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1.

b. ¿Qué versión de HTTP está ejecutando el servidor?

HTTP/1.1. La línea de estado de la respuesta (paquete 4302) indica HTTP/1.1 200 OK. En el detalle del paquete también se identifica que el servidor corre Apache/2.4.6.

c. ¿Qué lenguajes indica el navegador que acepta?

El navegador lo comunica mediante el encabezado Accept-Language dentro del GET. Los idiomas declarados fueron:

Accept-Language: es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7

Este encabezado permite al servidor decidir en qué idioma servir el contenido cuando existen varias versiones disponibles; los valores q indican el orden de preferencia.

d. ¿Cuántos bytes de contenido devolvió el servidor?

El encabezado Content-Length de la respuesta indica el tamaño del cuerpo del mensaje, sin contar los encabezados HTTP:

```
Content-Length: 81 bytes
```

Este valor corresponde únicamente al contenido HTML devuelto. Es importante no confundirlo con la longitud total del frame (495 bytes en el paquete 4302), que incluye además los encabezados de Ethernet, IP, TCP y HTTP.

e. ¿Dónde conviene "escuchar" los paquetes ante un problema de rendimiento? ¿Conviene instalar Wireshark en el servidor?

Ante un problema de rendimiento lo primero es determinar en qué punto del recorrido se está introduciendo el retardo, y para eso se necesita capturar en más de un lugar. Conviene escuchar en el cliente, para ver el tráfico tal como lo percibe el usuario final; en el router o gateway de salida de la red local, para distinguir si el problema es interno de la red o viene de afuera; y en el enlace WAN, para medir la latencia hacia el servidor.

Sobre instalar Wireshark en el servidor: técnicamente es la posición más útil, ya que permite ver el tráfico en el origen y comparar los tiempos de respuesta con lo que se observa en el cliente. Comparando ambas capturas se puede aislar si el retardo lo genera el servidor al procesar la petición o si se introduce en la red intermedia.

Sin embargo, en la práctica no siempre es conveniente ni posible. Wireshark en un servidor de producción consume CPU y memoria, lo que puede agravar el mismo problema de rendimiento que se está intentando diagnosticar. Además, en un servidor con mucho tráfico se genera un volumen enorme de datos, y suele haber restricciones de seguridad y de acceso administrativo. Por eso en producción se prefiere usar tcpdump, que es mucho más liviano y no requiere interfaz gráfica, y luego analizar el archivo resultante con Wireshark en otra máquina. Otra alternativa es configurar un port mirroring (SPAN) en el switch, lo que permite copiar el tráfico del servidor a un equipo dedicado sin tocar el servidor en absoluto.

6. Discusión

La primera parte del laboratorio dejó claro lo frágil que es la comunicación cuando no hay un protocolo bien definido. Transmitir en Morse y en Baudot a mano hace evidente algo que normalmente damos por sentado: que emisor y receptor tienen que estar de acuerdo en cómo se representa cada símbolo, dónde empieza y termina un mensaje, y qué hacer cuando algo sale mal. El problema de no poder distinguir los ceros en Baudot es exactamente el tipo de ambigüedad que los protocolos reales se esfuerzan en eliminar.

La segunda parte fue la contraparte: ver ese mismo problema ya resuelto, funcionando a escala y a una velocidad imposible de seguir manualmente. En la captura del archivo trace1 aparecen retransmisiones TCP, ACKs duplicados y segmentos perdidos — es decir, los mismos errores que nosotros cometíamos a mano, pero detectados y corregidos automáticamente por el protocolo. Ver los Dup ACK y los "previous segment not captured" después de haber sufrido el problema en carne propia le da un sentido distinto a la teoría.

Algo que llamó la atención al analizar el tráfico HTTP fue la cantidad de información que se puede extraer de una simple visita a una página. Solo con el GET y la respuesta ya se conoce la versión del protocolo, el software del servidor, el idioma preferido del usuario, el tamaño exacto del contenido y las direcciones de ambos

extremos. Y esto en HTTP plano, sin cifrar: en el resto de la captura, donde el tráfico va sobre TLS, ese contenido ya no es legible, lo cual explica muy bien por qué HTTPS se volvió el estándar.

Lo más útil de la herramienta, más allá del análisis puntual, fue entender la lógica de los filtros. Poder pasar de 7,035 paquetes a los 10 que realmente importan escribiendo una sola expresión es lo que hace que el análisis sea viable; sin filtros, la captura es simplemente ruido.

7. Conclusiones

- Los filtros son lo que hace usable a Wireshark. Una captura de miles de paquetes es inmanejable a menos que se sepa qué se está buscando y cómo expresarlo.
- El ring buffer resuelve un problema práctico real: permite capturar durante largos periodos sin llenar el disco, conservando siempre la ventana de tráfico más reciente, que es la que interesa cuando se espera un fallo intermitente.
- El tráfico HTTP sin cifrar expone bastante más información de la que uno imagina — versión, servidor, idioma, tamaño del contenido — lo que justifica de forma clara la adopción generalizada de HTTPS.
- Para diagnosticar un problema de rendimiento no basta con capturar en un solo punto: hace falta comparar capturas en distintos puntos del recorrido para aislar dónde se introduce el retardo.
- Instalar Wireshark en un servidor de producción es técnicamente valioso pero operativamente riesgoso; tcpdump o un port mirroring son alternativas más adecuadas.
- Ambas partes del laboratorio se complementan: la primera muestra por qué los protocolos existen, y la segunda muestra cómo se ven esos protocolos ya funcionando en una red real.

8. Referencias

Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.

Wireshark Foundation. Wireshark User's Guide. https://www.wireshark.org/docs/wsug_html_chunked/

Wireshark Foundation. Display Filter Reference. <https://www.wireshark.org/docs/dfref/>

Fielding, R., & Reschke, J. (2014). RFC 7231 — HTTP/1.1: Semantics and Content. IETF.

Wireshark Labs. Introduction to Wireshark. <http://gaia.cs.umass.edu/wireshark-labs/>